Variable

Variable is the container in which we can store values.

There are 3 ways of declaring variable:

1. `var`
2. `let`
3. `const`

`var`	`let`	`const`
Can be declared without assignment.	Can be declared without assignment.	Cannot be declared without assignment.
Re-declaration and re-assignment is possible.	Re-declaration is not possible but re-assignment is possible.	Neither re-claration nor re-assignment is possible.
Can be accessed before declaration.	Cannot be accessed before declaration.	Cannot beaccess before declaration
It has global scope.	It has block scope.	It has block scope.

Practice

```
// Redeclaration and re-assignment.

var a = 10
var a = 20         //re-declaring possible.
console.log(a)

let b = 30
//let b = 40        //No re-declaration. (SyntaxError: Identifier 'b' has
already been declared.)
b = 40
console.log(b)      //re-assignmet possible.


//const e               //No declaration without assignment. (SyntaxError:
Missing initializer in const declaration)
 const e = 70
 console.log(e)

//const e = 80          //No re-declaration. (SyntaxError: Identifier 'e' has
already been declared)
//e = 80                //No re-assignment. (TypeError: Assignment to constant
variable.)
```

```
// Accessing before declaration

console.log(c)         // result: undefined, Accessing possible.
var c = 50

//console.log(d)        // Accessing not possible. (ReferenceError: Cannot
access 'd' before initialization)
let d = 60


//console.log(f)         //Accessing not possible. (ReferenceError: Cannot
access 'f' before initialization)
const f = 90


//Scoping

{
var g = 100
let h=110
const i = 120
console.log(h, i)    // has Block access

}

console.log(g)       //has Global access
console.log(h)       //No global access.  (ReferenceError: h is not defined)
// console.log(i)    //No global access.  (ReferenceError: h is not defined)

console.log(window)    // g variable is present in the window, bec its has
Global scope
```

Datatypes

Premitive datatypes

1. String
2. Number
3. BigInt
4. Bool
5. Null
6. Undefined
7. Symbol

Non-premitive datatypes

1. Array
2. Object

3. Function
4. Set
5. Date
6. Map

Practice

```
//DATATYPES

// PREMITIVE DATATYPES:
console.log("PREMITIVE DATATYPES")

//1. String
console.log("\n")

var name1 = "jilani"
let name2 = "sagarmatha"
console.log(name1, typeof name1)
console.log(name2, typeof name2)


//2. Number
console.log("\n")

var name = 30
console.log(name, typeof name)     //output: 30 string, it gives wrong
datatypes when declared with 'var' and used 'name' as the variable name

// let name = 30
// console.log(name, typeof name)     //output: 30 'number', it gives correct
datatype when declared with 'let'

var num = 40
console.log(num, typeof num)

let num2 = -50
console.log(num2, typeof num2)

let num3 = 50.55
console.log(num3, typeof num3)

let num4 = -70.77
console.log(num4, typeof num4)

let num5 = 0.00
console.log(num5, typeof num5)           //output: 0 'number', the decimal
after 0 is automatically removed


//3. BigInt
console.log("\n")
```

```
var int = 40n
let int2 = 50n
console.log(int, typeof int)
console.log(int2, typeof int2)


//4. Bool
console.log("\n")

var flag = true
let flag2 = false
console.log(flag, typeof flag, flag2, typeof flag2)


//5. Null
console.log("\n")

var val = null
let val2 = null
console.log(val, typeof val, val2, typeof val2)                    //output: null
'object' null 'object', It shows datatype 'object' not null.


//6. Undefined
console.log("\n")

var value;
console.log(value)      //output: undefined

var value = undefined
console.log(value)

let value2 = undefined
console.log(value2)


// 7. Symbol
// Used to define unique value
console.log("\n")

var sym = Symbol("spiderman")
console.log(sym, typeof sym)          //output: Symbol(spiderman) 'symbol'



// REFERENCE DATATYPES:
console.log("\n")
console.log("REFERENCE DATATYPES")

//1. Array
// Used to store multiple data in one varible.
// It can store heterogenous data.
// It doesn't store the actual value.
// It store the memory address/reference of that values.
```

```
// We cannot assign/store multiple values to a single varible without using
array.

let arr = [1, 2, 3, "ram", true, null, 4.4]
console.log(arr, typeof arr)
console.log(arr[3])                 //indexing can be done on array
//console.log(arr[2:5])               //slicing cannot be done here


//2. Object
// It can store values in key-value pair just like dictionary of python

let obj = {
    name:"ram",
    address:"Lalitpur",
    roll:44                      //No need to provide quote for keys name while
deifning
}

console.log(obj)
console.log(obj[0])             //undefined, bec it will look for "0" key not
"0th" index
console.log(obj["roll"])        //44, Needs to provide quote while accessing
console.log(obj.address)        //Lalitpur, Can be accessed through dot notaion
also


//3. Function
// Function is a reusable block of code which can perform a specific task when
called
// Syntax: function functionName(){}

function hello(){
    console.log("Hello JS!")
}
hello()


// Parameter and Argument in Function

function add(num1, num2){
    console.log(num1+num2)
}
add(10, 20)

// Return in Function

function multiply(num1, num2){
    return num1*num2
    console.log("After return...")        //this will not be printed bec the
flow of control returns to the place of call when it executes the 'return'
statement. No code is executed after 'return' statement.
}
console.log(multiply(2,3))
```

```
console.log(multiply(2,3))
console.log(multiply(2,3))                //this will print it multiple times.
the codes is executed the number of times it is called
result = multiply(4,4)                    //storing result in a variable
console.log(result)
```